Java One Cool Cat (Meow Facts) API Tutorial

Contents

Java One Cool Cat (Meow Facts) API Tutorial 1
What is an API? 1
Why Use a Web API? 3
Public API's 3
Meow Facts API 3
What is JSON? 3
Jackson JSON Libraries 4
JSON to POJO - MeowData 6
MeowFacts Java App 8
Time for a Meow Fact 9
Assignment Submission 10

This tutorial was inspired by a student screencast submission.

Time required: 60 minutes

- Comment each line of code as shown in the tutorials and other code examples.
- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.
- Please read all the directions before beginning the assignment.

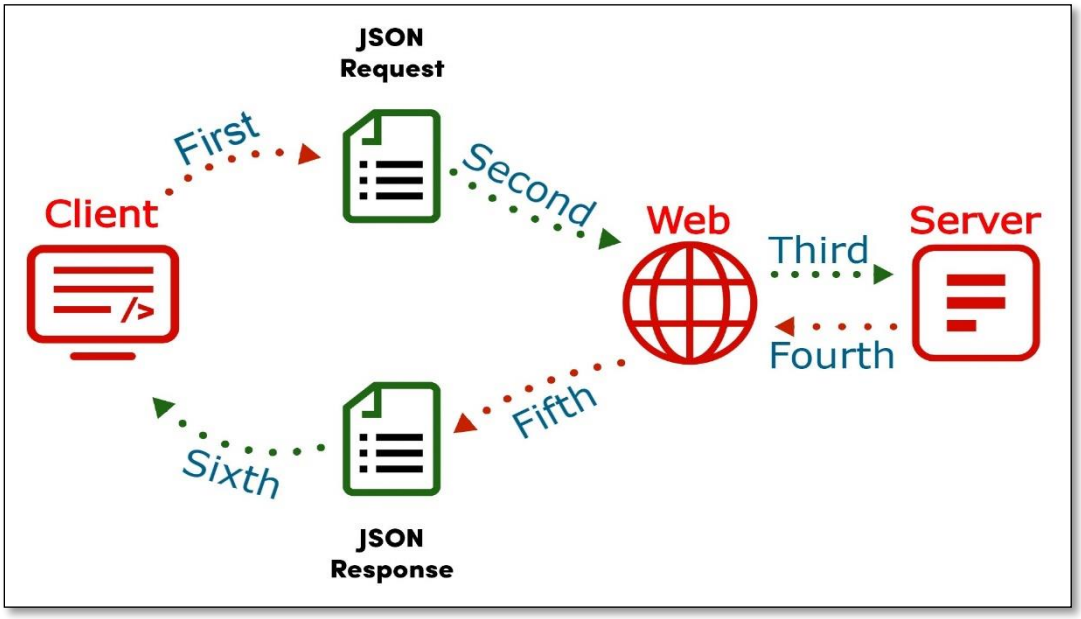
What is an API?

What is an API? This is what Wikipedia says.

"An **application programming interface** ('API') is a computing interface that defines interactions between multiple software intermediaries. It defines the kinds of calls or requests that can be made, how to make them, the data formats that should be used, the conventions to follow, etc. It can also provide extension mechanisms so that users can extend existing functionality in various ways and to varying degrees. An API can be entirely custom, specific to a component, or designed based on an industry-standard to ensure

interoperability. Through information hiding, APIs enable modular programming, allowing users to use the interface independently of the implementation."

An API is a software intermediary that allows two applications to talk to each other. In other words, an API is the messenger that delivers your request to the provider that you're requesting it from and then delivers the response back to you.



Web API's are hosted on webservers. Instead of a web browser asking for a webpage (like how most interactions with the internet is done today) your program asks for data. This data is usually returned to your program in a JSON format. JSON stands for JavaScript Object Notation and is a lightweight text-based data-interchange format.

To get data to our program we make a request to a web server. The webserver then replies with our desired data and a status code. Sometimes an API key is needed to be sent along with the request to successfully return data.

There are many ways to call information from an API. One of the most common are **get** requests. You use a get request each time you browse to a web site.

For example, to request data in a Python script using the extra functionality gained from the request library the statement below would be used.

```
response = requests.get('http://api_example.com/example.json')
```

This data object can be named anything, response was used as the identifier for this example. This API doesn't exist. This is the format they tend to take.

Data from the web can tell us the exact weather situation of any place on the globe, the location of faces on an image we provide or the number of astronauts currently floating far, far out in space. Any information freely accessible on the World Wide Web is literally within our grasp from within our programs.

Why Use a Web API?

You could use static data which you have downloaded or created yourself. Data used this way with Java would work perfectly fine. Here are some examples of when APIs are more useful than static data sets.

- When the data is changing quickly, and you need the data to be accurate and up to date. Planes are currently in the air or the location of the international space station, the data is constantly changing.
- When only a small piece of a much larger data set is desired.
- When you are taking advantage of already completed, repeated computations. Sources such Facebook, Spotify and Google already have a ton of data, which have undergone serious calculations, which you can access easily with an API request.

In cases like the ones above, a Web API is the right solution. Using an API will save time and effort over doing all the computation ourselves or constantly refreshing our desired data.

Public API's

There are hundreds of public API's available. This website contains a list of some of them.

https://github.com/public-apis/public-apis

Meow Facts API

Who doesn't want to know random facts about cats?

We do! Let's create a Java program to help us with our cat education.

Before we do that, we have a few new concepts to go through.

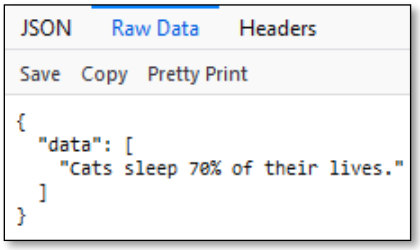
What is JSON?

The Meow Facts API is a free https-based Web API which provides a JSON response.

Raw API data is typically in a text file in JSON or XML format. JSON is the most popular format for web API's.

JavaScript Object Notation (JSON) is an open standard text-based file and data interchange format. It uses human-readable text to store and transmit data objects consisting of attribute–value pairs and array data types.

Go to https://meowfacts.herokuapp.com to see some raw JSON data. This view is using Firefox.



Jackson JSON Libraries

This will be a Visual Studio Code project in Java.

We will be retrieving data from the web in JSON format. Java doesn't have any native JSON parsing or handy dandy requests library like Python.

1. Create a folder to hold your project: **MeowFacts**
2. Open the MeowFacts folder with Visual Studio Code.
3. Create two folders in your **MeowFacts** program folder.
    a. **lib**
    b. **src**

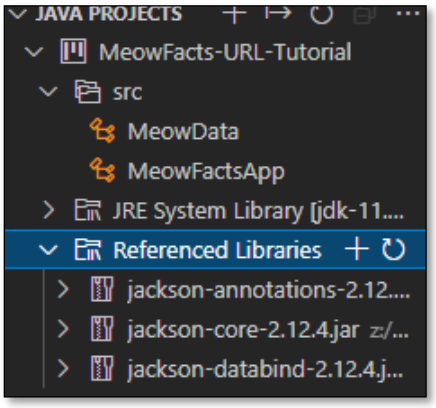
We are going to add three libraries to parse out the JSON data into Java classes.

4. Download these three files.
    a. jackson-annotations-2.16.1.jar
    b. jackson-core-2.16.1.jar
    c. jackson-databind-2.16.1.jar
5. Copy and paste the three jackson jar files into the **lib** folder.
6. In the **src** folder, create two files.

7. **MeowFactsApp.java** and **MeowData.java**

8. Double Click **MeowFactsApp.java**

9. You should see the following at the bottom left if you open **Java Projects → Referenced Libraries**



JSON to POJO - MeowData

JSON stands for JavaScript Object Notation, a lightweight data-interchange format. To convert JSON (JavaScript Object Notation) to POJO (Plain Old Java Object) in Java, follow these steps:

1. Create a Java class: Define a Java class that mirrors the structure of the JSON data. Include fields that match the JSON object's properties.
2. Use a library: Libraries like Jackson, Gson, or org.json can parse JSON to Java objects automatically by mapping JSON keys to Java object fields.

There are many JSON to POJO converters on the web.

Go to https://meowfacts.herokuapp.com to see some raw JSON data. This view is using Firefox.

```
{
    "data": [
        "Besides smelling with their nose, cats can smell with an
additional organ called the Jacobson's organ, located in the upper
surface of the mouth."
    ]
}
```

https://json2csharp.com/code-converters/json-to-pojo

This is the result after using the above web site with properties checked.

```
// import com.fasterxml.jackson.databind.ObjectMapper;
// import com.fasterxml.jackson.annotation.JsonProperty;
/* ObjectMapper om = new ObjectMapper();
Root root = om.readValue(myJsonString, Root.class); */
public class Root{
    @JsonProperty("data")
    public ArrayList<String> getData() {
            return this.data; }
    public void setData(ArrayList<String> data) {
            this.data = data;}
    ArrayList<String> data;
}
```

To convert the JSON data from the API into a Plain Old Java Object, we create a class modeled after this conversion. In this instance, an Array works as well as an ArrayList.

Enter the following code into **MeowData.java**

```java
/*************************************************************
* Filename:    MeowData.java
* Written by:
* Written on:
* Revised:
* Description: Convert data fetched from MeowAPI endpoint
*************************************************************/
public class MeowData {
    /**
     * Meow Fact data member
     */
    private String data;

    /**
     * Get fact
     */
    public String getData() {
        return this.data;
    }

    /**
     * Set the json array data into a string
     */
    public void setData(String[] data) {
        this.data = data[0];
    }

    /**
     * Override system object to String method with User-friendly String
     * representation of object
     */
    @Override
    public String toString() {
        String output = String.format("\nMeow Fact: %s", this.data);
        return output;
    }
}
```

MeowFacts Java App

The main program uses the **MeowData** class to convert the incoming JSON to an object. We can access API data members like any other object.

Add the following code to the **MeowFactsApp.java** file.

```java
/*************************************************************
* Filename: MeowFactsApp.java
* Written by:
* Written on:
* Revised:
* Description: Get Meow Facts from Meowfacts API endpoint
*************************************************************/

// Handle any Input Output exceptions
import java.io.IOException;
// Import URL library
import java.net.URL;
// Import Jackson JSON libraries
import com.fasterxml.jackson.core.JsonGenerationException;
import com.fasterxml.jackson.databind.JsonMappingException;
import com.fasterxml.jackson.databind.ObjectMapper;

public class MeowFactsApp {

    // Web API endpoint URL
    private final static String API_ENDPOINT = "https://meowfacts.herokuapp.com/";
    // Create MeowData instance
    private static MeowData fact;

    public static void main(String[] args) {
        // Get data from Web API
        getData();
        // Display a MeowFact at the console
        System.out.println(fact.toString());
    }

```

```java
    /**
     * Get data from the API endpoint
     */
    public static void getData() {
        try {
            // Create URL object
            URL url = new URL(API_ENDPOINT);

            // Create Jackson JSON object to read the raw JSON data
            // into POJO (Plain Old Java Objects)
            ObjectMapper objectMapper = new ObjectMapper();

            // Read JSON source into MeowData Object
            fact = objectMapper.readValue(url, MeowData.class);

            // JSON Exception handling
            // e.printStackTrace() will print the exception
        } catch (JsonGenerationException | JsonMappingException e) {
            e.printStackTrace();
            // Input Output Exception handling
        } catch (IOException e) {
            e.printStackTrace();
            // Handle any other exceptions
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

The above code does 3 things.

1. Creates a Jackson object to read JSON into POJO (Plain Old Java Objects).
2. Reads the JSON into a **MeowFact** object.
3. Displays a Meow Fact.

Time for a Meow Fact

Time to run your new application.

Example run:

```
Meow Fact: Cats can be taught to walk on a leash, but a lot of time
and patience is required to teach them. The younger the cat is, the
easier it will be for them to learn.
```

Assignment Submission

1. Insert a screenshot of a successful run of the program.
2. Zip up the MeowFacts folder.
3. Attach the zip file to the assignment in Blackboard.